CAHIER DES CHARGES

Human Machine Lab

Plateforme modulaire pour transformer des besoins en spécifications et prototypes vérifiables

Version : 1.0 — proposition de cadrage

Date : 13 octobre 2026

Stack : Laravel · Alpine.js · Vue.js · Reka-UI · Hostinger Startup Cloud

Document de travail destiné à guider le développement, la sécurisation, le déploiement et la présentation du projet dans un CV ou un portfolio.

1. Résumé exécutif

Human Machine Lab est un laboratoire logiciel qui permet de transformer un besoin métier exprimé en langage naturel en une spécification structurée, puis en un prototype ou une tâche de développement vérifiable. Le projet doit d'abord prouver un parcours simple et fiable ; il ne doit pas tenter de devenir immédiatement une plateforme universelle d'agents IA.

Le produit initial recommandé est un outil de cadrage de demandes : l'utilisateur décrit un problème, complète les informations manquantes, obtient une fiche structurée (objectif, utilisateurs, contraintes, règles métier, critères d'acceptation) et peut suivre l'état de traitement. L'IA est facultative et doit rester contrôlée ; la première version doit fonctionner sans dépendre d'un modèle payant.

Objectif double : (1) disposer d'un projet démontrable pour les candidatures en développement web/logiciel, qualité logicielle ou intégration IA ; (2) tester des besoins réels qui pourraient ensuite donner naissance à des produits spécialisés. Les produits spécialisés (pâtisserie, événementiel, candidatures, test d'agents) restent séparés tant que leur marché n'est pas validé.

2. Objectifs et hors périmètre

2.1 Objectifs

- Livrer une application fonctionnelle de bout en bout, déployée en test et en production.
- Montrer une architecture Laravel claire : routes, contrôleurs, Form Requests, modèles Eloquent, migrations, politiques d'accès, tâches asynchrones et tests.
- Démontrer une interface cohérente : Blade et Alpine.js pour les pages simples ; Vue.js et Reka-UI pour les interactions riches lorsque cela apporte une valeur réelle.
- Réutiliser les composants, conventions, modèles de données et mécanismes de jetons déjà présents, après audit de leur fonctionnement et de leur sécurité.
- Préparer un cas d'étude crédible pour le CV : problème, décisions, architecture, tests, sécurité, déploiement et limites connues.
- Garder une voie d'évolution vers des produits payants, sans ajouter prématurément des fonctionnalités coûteuses.

2.2 Hors périmètre du MVP

- Créer automatiquement une application complète à partir d'un prompt.
- Orchestrer un réseau complexe d'agents autonomes ou promettre de tester « tous les agents ».
- Créer dès le départ plusieurs produits métier dans la même interface.
- Stocker des CV, offres confidentielles ou données client sensibles sans besoin défini ni politique de conservation.
- Ajouter paiements, abonnements, marketplace ou fonctionnalités sociales avant validation du besoin.

3. Utilisateurs et parcours principal

3.1 Utilisateurs

- Visiteur : comprend la proposition, consulte une démonstration et peut soumettre un besoin sans fournir de données sensibles.
- Utilisateur connecté (phase ultérieure ou MVP si nécessaire) : crée et retrouve ses demandes et leurs spécifications.
- Administrateur : consulte les états opérationnels nécessaires, traite les signalements et gère la configuration sans accès excessif au contenu privé.

3.2 Parcours MVP

Étape	Comportement attendu	Résultat vérifiable
1. Décrire	L'utilisateur saisit un problème, le contexte et le résultat attendu.	Demande créée avec validation serveur.
2. Structurer	Le système normalise les champs et identifie les informations manquantes.	Fiche avec objectif, acteurs, contraintes et règles.
3. Définir les tests	L'utilisateur ou le système formule des critères d'acceptation.	Critères observables et testables.
4. Suivre	L'utilisateur voit l'état de la demande.	Brouillon, prête à vérifier, validée ou archivée.
5. Exporter	L'utilisateur exporte ou copie sa fiche.	Document lisible, sans fuite de données d'autres utilisateurs.

4. Fonctionnalités

4.1 MVP obligatoire

- Landing page présentant le problème résolu, le fonctionnement, une démonstration et un appel à l'action clair.
- Formulaire de création de demande avec validation serveur, limites de taille et messages d'erreur utiles.
- Modèle de données structuré : titre, description, objectif, utilisateurs concernés, contraintes, règles, critères d'acceptation et statut.
- Écran de résultat permettant de consulter, modifier et valider une spécification.
- Gestion des états et des erreurs ; aucune promesse de traitement asynchrone si aucun worker n'est configuré.
- Composants d'interface réutilisables avec règles d'accessibilité et états chargement/erreur/vide.
- Tests automatisés des règles métier, de la validation, des autorisations et des parcours essentiels.
- README technique, procédure d'installation, configuration par environnement et procédure de déploiement/retour arrière.

4.2 Évolution après validation du MVP

- V1.1 — Comptes : authentification, espace personnel et politiques d'accès si la sauvegarde privée est nécessaire.
- V1.2 — Tâches asynchrones : lancer une génération ou une analyse en arrière-plan, suivre l'état (en attente, en cours, terminé, échoué, annulé), relancer de façon contrôlée et notifier l'interface.
- V1.3 — Assistance IA : proposer une spécification depuis du texte libre, afficher les champs incertains, permettre la correction humaine et mesurer les erreurs.
- V1.4 — Évaluation d'agents : scénarios de test, jeux d'exemples, critères d'évaluation, rapports et tests de régression.
- V2 — Extraction d'un produit : choisir un seul cas métier récurrent sur la base d'entretiens, d'usage réel et de volonté de payer.

5. Architecture technique cible

5.1 Principes

- Laravel reste la source de vérité côté serveur : règles métier, validation, autorisations, persistance et orchestration.

- Blade sert les pages publiques et les écrans simples ; Alpine.js ajoute de petites interactions locales.
- Vue.js est réservé aux interfaces plus riches (éditeur structuré, suivi de tâches, comparaison de résultats). Reka-UI fournit des primitives de composants accessibles ; vérifier la compatibilité exacte avec la version de Vue et les dépendances retenues.
- Ne pas introduire un framework ou une couche d'abstraction supplémentaire sans besoin démontré.
- Utiliser Form Requests pour la validation, Policies/Gates pour l'autorisation et des services/actions seulement lorsqu'ils clarifient une logique métier réellement réutilisée.
- Les migrations doivent être versionnées, relues et exécutées dans le cadre du déploiement ; ne pas modifier directement la base de production.
- Le système de jetons existant doit être audité avant réutilisation : nature du jeton, finalité, portée, durée de vie, stockage, révocation et journalisation.

5.2 Schéma logique indicatif

Élément	Responsabilité
Web / UI	Blade, Alpine.js, Vue.js et Reka-UI ; affichage et interaction.
Application Laravel	Contrôleurs minces, Form Requests, Policies, logique métier.
Données	Modèles Eloquent et migrations ; contraintes et relations cohérentes.
Jobs / Queue (phase asynchrone)	Traitements hors requête HTTP, tentatives limitées, délais et états.
Services externes (optionnels)	Fournisseur de modèle IA ou API ; accès via une couche isolée et configuration serveur.
Observabilité	Logs techniques sans secrets ni contenu privé inutile, suivi des erreurs et alertes adaptées.

Le choix de base de données, du driver de queue, du cache et du stockage doit être confirmé avec les capacités et limites du plan Hostinger Startup Cloud. Ne pas supposer qu'un worker persistant, un service supervisé ou un accès système particulier est disponible sans le vérifier dans le panneau d'hébergement et la documentation de l'offre.

6. Modèle de données indicatif

Les noms ci-dessous sont une proposition à comparer aux modèles et migrations existants. Éviter de créer des doublons : faire d'abord un inventaire du schéma actuel et de son historique.

Entité	Champs / règles principaux
users (si comptes)	Identité minimale, email vérifié si requis, mot de passe hashé, timestamps.
requests	id, user_id nullable selon le mode d'accès, title, description, status, timestamps.
specifications	request_id, goal, actors, constraints, business_rules, version, validated_at.
acceptance_criteria	specification_id, description, expected_result, position, status.
background_tasks (phase 1.2)	type, owner_id, status, input_reference, result_reference, attempts, started_at, finished_at, error_code.
audit_events (sélectif)	actor_id, action, target_type/id, timestamp, métadonnées minimales ; jamais de secret.

Entité	Champs / règles principaux
personal_access_tokens (si Laravel Sanctum)	Utiliser le schéma et le comportement standards du package, scopes limités et révocation ; ne pas stocker de jeton en clair.

- Définir clés étrangères, index, contraintes de statut et règles de suppression avant la migration.
- Éviter les champs JSON fourre-tout pour des données qui doivent être filtrées, validées ou référencées ; réserver JSON aux structures réellement flexibles.
- Prévoir la suppression d'une demande et de ses données liées, avec une politique de rétention explicite.
- Les données de production ne doivent pas être copiées telles quelles dans l'environnement de test.

7. Environnements et déploiement Hostinger

7.1 Séparation test / production

Domaine	Règles
Test / staging	Configuration et secrets propres, base séparée, comptes de démonstration synthétiques, envoi email neutralisé ou limité, indexation désactivée.
Production	Domaine public, base et secrets distincts, HTTPS, sauvegardes, accès administratif restreint, logs surveillés.
Local	Variables locales non versionnées, données synthétiques, tests automatisés et procédure reproductible.

7.2 Pipeline de livraison recommandé

- Travailler dans une branche de fonctionnalité ; revue du diff et tests avant fusion.
- Exécuter les tests, le lint/format si configuré, et vérifier les dépendances vulnérables.
- Déployer d'abord en staging : sauvegarde si nécessaire, installation des dépendances de production, build des assets et migrations compatibles.
- Effectuer un smoke test : page d'accueil, création d'une demande, autorisations, logs et tâches de fond si activées.
- Promouvoir le même artefact ou le même commit vers la production ; ne pas reconstruire manuellement une version différente.
- Préparer un retour arrière applicatif et une stratégie de restauration de base. Une migration destructive peut nécessiter une procédure spécifique.
- Documenter la façon de lancer les migrations, vider les caches, redémarrer les workers si le plan les permet et vérifier le résultat.

Sur Hostinger Startup Cloud, confirmer explicitement les possibilités de déploiement automatisé, de cron, de workers persistants, de sauvegarde/restauration et les limites CPU/RAM/stockage. Le plan doit être conçu selon les fonctions réellement disponibles, sans supposer l'accès à un VPS root ou à un orchestrateur.

8. Sécurité et protection des données

8.1 Principes

- Minimisation : collecter seulement les informations nécessaires au cas d'usage. Ne pas demander de CV, données de santé, données client ou secrets si ce n'est pas indispensable.
- Contrôle d'accès : vérifier côté serveur, pour chaque lecture, modification, export et suppression, que l'utilisateur a le droit d'accéder à la ressource. Ne jamais se fier uniquement à l'interface.

- Validation : valider toutes les entrées côté serveur, limiter tailles et formats, échapper les sorties et prévenir les injections SQL/XSS via les pratiques Laravel.
- Authentification : utiliser les mécanismes Laravel éprouvés, mots de passe hashés, protection contre les tentatives répétées et MFA pour les comptes administratifs si possible.
- Sessions et CSRF : cookies sécurisés, HttpOnly, Secure en HTTPS, SameSite adapté, rotation de session et protection CSRF pour les opérations web.
- Jetons : distinguer les jetons de session, API, invitation ou accès temporaire ; limiter leur portée, durée et usage ; les stocker sous forme hashée si le mécanisme le permet ; les révoquer ; ne jamais les exposer dans l'URL, les logs ou le dépôt Git.
- Secrets : clés d'API et secrets uniquement dans la configuration serveur / variables d'environnement, jamais dans le JavaScript livré au navigateur ou dans le dépôt.
- Fichiers : si des pièces jointes sont ajoutées, limiter type et taille, renommer côté serveur, contrôler l'accès et éviter l'exécution de fichiers téléversés.
- IA externe : informer l'utilisateur des données envoyées à un fournisseur, réduire ou anonymiser les données, vérifier les conditions de conservation et ne pas transmettre de données sensibles par défaut.
- Journalisation : ne pas enregistrer mots de passe, jetons, clés, prompts privés ou documents complets par défaut ; utiliser des identifiants de corrélation et des codes d'erreur.

8.2 GDPR / RGPD et obligations en Allemagne

- Définir le responsable du traitement, les finalités, la base juridique, les destinataires, les durées de conservation et les modalités d'exercice des droits avant d'ouvrir le service au public.
- Publier une politique de confidentialité adaptée au traitement réel. Une simple mention générique ne suffit pas.
- Prévoir un Impressum conforme aux obligations applicables au service et à son exploitant. La nécessité et le contenu exacts dépendent notamment de la nature de l'offre et de son caractère commercial ; faire vérifier la version finale.
- Utiliser des cookies non essentiels ou des outils de suivi uniquement après avoir vérifié les obligations de consentement applicables.
- Si un fournisseur IA ou d'hébergement traite des données pour le compte de l'exploitant, examiner les accords de traitement, les transferts et les sous-traitants pertinents.
- Documenter les procédures de demande d'accès/suppression, les incidents de sécurité et la gestion d'une éventuelle violation de données.

Ce cahier des charges est une orientation d'ingénierie, pas un avis juridique. Les obligations exactes dépendent de l'exploitant, des fonctionnalités, des données et du modèle économique ; faire vérifier les textes juridiques avant la mise en ligne publique.

9. Landing page, Impressum ou application seule ?

Élément	Priorité	Recommandation
Application fonctionnelle	Indispensable	C’est la preuve principale pour un recruteur : un parcours complet, testé et démontrable.
Landing page	Haute	Une page simple clarifie le problème, la valeur, le fonctionnement et fournit un accès à la démo.
Impressum	Avant lancement public si applicable	Nécessaire selon les obligations applicables en Allemagne ; préparer une page distincte et exacte.
Datenschutzerklärung	Avant collecte publique	À prévoir dès qu’il y a traitement de données personnelles ; refléter les traitements réels.
Page technique / case study	Haute pour le CV	Présenter architecture, choix, tests, sécurité, déploiement, captures et limites.
Blog / newsletter / tarifs	Plus tard	Seulement si cela sert une hypothèse de distribution ou de monétisation validée.

Conclusion : pour un CV, un produit fonctionnel peut suffire, à condition qu’il soit reproductible, documenté et testable. Pour attirer aussi des clients et partenaires, j’ajouterais une landing page légère et une étude de cas. L’Impressum et la politique de confidentialité sont des sujets de conformité à traiter avant la mise en ligne publique lorsque requis, pas des fonctionnalités qui prouvent à elles seules la qualité technique.

10. Méthode de développement

Phase 0 — Audit avant toute modification

- Cartographier les routes, composants Blade/Vue/Alpine, modèles, migrations, système de jetons, dépendances et tâches existantes.
- Vérifier la séparation des environnements, les secrets, les permissions de fichiers, les sauvegardes et les chemins de déploiement.
- Identifier les doublons et la dette technique ; ne pas réécrire les composants qui fonctionnent sans raison mesurable.
- Écrire une courte ADR (Architecture Decision Record) pour les décisions importantes.

Phase 1 — Vertical slice fonctionnelle

- Livrer le parcours complet : saisir une demande → la valider → enregistrer → afficher → modifier → exporter.
- Ajouter tests unitaires et feature tests sur les règles métier et les permissions.
- Faire une démonstration avec des données synthétiques et mesurer le temps de réalisation et les points de friction.

Phase 2 — Durcissement et staging

- Vérifier la sécurité, les erreurs, les limites de saisie, la gestion des accès et les suppressions.
- Déployer en staging avec configuration séparée ; documenter une procédure de test de recette.
- Exécuter les smoke tests et tester la restauration ou le retour arrière dans la mesure permise par l’hébergement.

Phase 3 — Publication et preuve

- Mettre en ligne une landing page, une démo contrôlée, l'Impressum et les informations de confidentialité nécessaires.
- Publier le README, le diagramme d'architecture, les décisions techniques et une courte étude de cas.
- Ne publier aucune donnée client réelle, aucun secret, aucune capture d'écran contenant des informations privées.

Phase 4 — Une seule extension à la fois

- Mesurer l'usage ou interroger des utilisateurs potentiels avant d'ajouter l'IA.
- Ajouter le traitement asynchrone seulement lorsqu'une tâche prend assez de temps ou doit continuer après la requête HTTP.
- Ajouter l'évaluation d'agents seulement si des scénarios d'agents existent à tester.
- Valider la volonté de payer avant de construire la facturation ou une plateforme multi-produit.

11. Stratégie pour le CV et le portfolio

11.1 Ce que le recruteur doit pouvoir vérifier

- Un lien de démonstration ou une vidéo courte montrant le parcours complet.
- Un README avec installation, architecture, variables nécessaires (sans secrets) et commandes de test.
- Des tests automatisés visibles et un résumé honnête de leur couverture.
- Un diagramme simple des composants et du flux de données.
- Une section sécurité : contrôles d'accès, gestion des jetons, secrets, minimisation des données et limites connues.
- Une section déploiement : staging/production, migrations, smoke tests, sauvegarde et retour arrière.
- Un exemple avant/après d'une demande transformée en spécification et critères d'acceptation.

11.2 Exemple de formulation CV — à utiliser seulement après réalisation

Human Machine Lab — Entwicklung einer modularen Webanwendung zur strukturierten Erfassung und Validierung von Anforderungen
Entwicklung mit Laravel, Blade/Alpine.js und Vue.js; Modellierung von Daten und Migrationen, Implementierung serverseitiger Validierung und Zugriffskontrollen, Aufbau automatisierter Tests sowie Trennung von Staging- und Produktionsumgebung. Vorbereitung einer erweiterbaren Architektur für asynchrone Aufgaben und kontrollierte KI-Integration.

Ne pas mentionner une fonctionnalité comme terminée tant qu'elle n'est pas réellement implémentée et testée. Les technologies réellement utilisées et les résultats mesurés doivent remplacer toute formulation générique.

11.3 Adapter la preuve au poste

- Web / Fullstack : montrer modèle de données, validation, autorisations, UX et tests.
- QA / Softwarequalität : montrer critères d'acceptation, tests de régression, rapports d'erreurs et procédure de reproduction.
- KI-Entwicklung : montrer un petit benchmark reproductible, un jeu d'exemples, les critères d'évaluation et les limites du modèle.
- Technical Presales : montrer le problème métier, une démonstration courte, les compromis techniques et une explication accessible.

12. Indicateurs de réussite

Dimension	Indicateur de preuve
Fonctionnel	Un utilisateur termine le parcours principal sans intervention manuelle du développeur.
Qualité	Les tests critiques passent en local et dans le pipeline de staging.
Sécurité	Les accès non autorisés sont rejetés par des tests ; aucun secret n’est présent dans le dépôt.
Déploiement	Un déploiement staging documenté et un retour arrière préparé.
Réutilisabilité	Au moins un composant commun est utilisé à plusieurs endroits sans duplication de logique.
Valeur métier	Retours documentés de quelques utilisateurs potentiels et décision explicite sur la prochaine fonctionnalité.
Portfolio	Démo, README, architecture, tests et étude de cas accessibles et cohérents.

13. Risques et décisions ouvertes

Risque / question	Mesure
Périmètre trop large	Limiter le MVP au parcours demande → spécification → critères → validation.
Sur-ingénierie	Ne pas construire un orchestrateur d’agents avant d’avoir un cas d’usage qui le justifie.
Coût IA imprévisible	MVP déterministe, appels IA optionnels, limites de taille, quotas et mesure du coût.
Limites de l’hébergement	Valider queue, cron, stockage, logs et déploiement selon l’offre Hostinger réelle.
Fuite de données	Séparation des environnements, accès côté serveur, données synthétiques en staging, rétention et logs minimaux.
Absence de marché	Entretiens et prototypes ciblés avant la monétisation SaaS.
Système de jetons mal compris	Inventorier le type de token et son usage avant toute réutilisation ou exposition.

14. Définition du MVP terminé

Le MVP peut être considéré comme présentable lorsque les points suivants sont vrais :

- Le parcours principal fonctionne sur un environnement de démonstration.
- Les règles de validation et d’autorisation sont testées.
- Les modèles et migrations sont cohérents avec les données manipulées.
- La configuration de staging est distincte de celle de production et aucun secret n’est versionné.
- Le déploiement et les migrations sont documentés ; les vérifications post-déploiement sont définies.
- Les pages publiques et les obligations de confidentialité/légales pertinentes sont traitées avant ouverture publique.
- La démonstration n’utilise que des données synthétiques ou autorisées.
- Le README et l’étude de cas décrivent honnêtement ce qui est terminé, ce qui reste à faire et les limites connues.

Fin du cahier des charges — Version 1.0. Ce document est une base de travail à confronter à l’état réel du dépôt, aux migrations existantes et aux capacités exactes du plan d’hébergement.